

Plan de Branching — SmartLogix

DSY1106 Desarrollo Fullstack III

Evaluación Parcial N°2

1. Estrategia adoptada: Git Flow simplificado

El proyecto utiliza una variante simplificada de **Git Flow**, adaptada al trabajo individual con múltiples microservicios.

2. Ramas principales

main

- Código estable. Solo recibe merges desde **develop** cuando un conjunto de funcionalidades está completo y probado.
- Protegida: no se hace commit directo.

develop

- Rama de integración. Concentra el trabajo de todos los microservicios backend.
- Es la rama activa de trabajo; todos los commits de funcionalidades se integran aquí.
- Se usa para construir y probar el sistema completo con Docker Compose.

frontend

- Rama dedicada al desarrollo de la aplicación Angular.
 - Separada de **develop** para evitar conflictos entre cambios de backend y frontend.
 - Se mergea a **main** cuando el frontend está estable.
-

3. Ramas de features (backend)

Rama	Propósito	Estado
feature/bff-gateway	Implementación del API Gateway con JWT	Mergeada a develop
feature/inventario-implementacion	Microservicio Inventario completo	Mergeada a develop
feature/producto-implementacion	Microservicio Producto completo	Mergeada a develop
feature/tests-unitarios	Tests JUnit 5 + Mockito para Orden	Mergeada a develop

4. Flujo de trabajo

```
feature/xxx → develop → main  
frontend   → main
```

Proceso:

1. Se crea una rama `feature/nombre-funcionalidad` desde `develop`
2. Se desarrolla y se hacen commits atómicos en la feature branch
3. Se hace merge a `develop` cuando la funcionalidad está lista
4. Cuando `develop` está estable y probado, se mergea a `main`

5. Convención de commits

Formato: `tipo(alcance): descripción breve`

Tipo	Uso
<code>feat</code>	Nueva funcionalidad
<code>fix</code>	Corrección de bug
<code>refactor</code>	Refactorización sin cambio de comportamiento
<code>test</code>	Agregar o corregir tests
<code>docs</code>	Documentación
<code>ci</code>	Cambios en pipelines CI/CD

Ejemplos del historial:

```
feat(security): RUT encriptado, privacidad por rol y rutas para transportista  
fix(orden): corregir error de tipos en UsersClient con Map<?,?>  
fix(ci): agregar settings.xml con repo Spring – corrige 403 en Spring Boot 4.0.5  
feat(orden): agregar OrdenFactory y OrdenFactoryTest que faltaban en git
```

6. Repositorio

URL: <https://github.com/benjazzx/SmartLogix>

Ramas remotas activas:

- `origin/main`
- `origin/develop`
- `origin/frontend`
- `origin/feature/bff-gateway`

- origin/feature/inventario-implementacion
 - origin/feature/producto-implementacion
 - origin/feature/tests-unitarios
-

7. Diagrama de ramas

